

MODULE 01 · EXAM STRATEGY - ALL DOMAINS

The mental model

How an AWS data pipeline fits together, and the three constraints that decide every question.

DOMAIN All MODULE 1 of 27 SCENARIOS 2 worked

On this exam, most options **would work**.
Only one fits the constraint.

Every pipeline has the same five stages

01

Ingest

Get data in.
Batch pull or
streaming push.

02

Store

Land it
somewhere.
Usually Amazon
S3.

03

Catalog

Record what it is,
where, and its
schema.

04

Transform

Clean, join,
reshape,
aggregate.

05

Serve

Query,
dashboard, or
feed another
system.

ON THE EXAM

Security and monitoring are not a sixth stage. They wrap all five.

The five stages map onto the four domains

- ◆ **Ingest and Transform** are Domain 1. The largest, at 34 percent.
- ◆ **Store and Catalog** are Domain 2, at 26 percent.
- ◆ **Serve, plus operating the whole thing**, is Domain 3, at 22 percent.
- ◆ **Everything wrapped around it** is Domain 4, at 18 percent.
- ◆ Orchestration sits oddly. It appears in Domain 1 and again in Domain 3.

ON THE EXAM

Orchestration is tested twice. Do not treat it as a minor topic.

Cost. Latency. **Operational overhead.**
Nearly every question turns on one of them.

The qualifier tells you which constraint

PHRASE IN THE QUESTION	WHAT IT IS REALLY ASKING
LEAST operational overhead	Choose the more managed or serverless service
LEAST development effort	Choose the one needing less custom code, not less admin
MOST cost-effective	Cheapest that still meets every stated requirement
LEAST latency, real time	Streaming, or an in-memory store. Not batch
Near real time	Seconds to minutes is acceptable. Firehose territory
Minimal changes to existing	Keep the current architecture. Do not re-platform

ON THE EXAM

Overhead and development effort are different axes. A service can be low in one and high in the other.

The overhead axis

YOU OPERATE IT

Provisioned and self-managed

- Amazon EMR on EC2, MSK provisioned
- You size, patch, scale and tune
- Cheaper at steady high utilisation
- Full control over versions and config

AWS OPERATES IT

Serverless and managed

- Glue, Athena, Lambda, EMR Serverless
- No capacity planning, pay per use
- Cheaper for spiky or unpredictable load
- Less tuning available when you need it

THE TIEBREAKER

Steady and predictable favours provisioned. Spiky or occasional favours serverless.

Where the data services actually sit

Roughly, from most managed to least

- ◆ **Fully serverless.** Athena, Lambda, Data Firehose, Glue, Step Functions.
- ◆ **Serverless variants.** Redshift Serverless, EMR Serverless, MSK Serverless, Aurora Serverless.
- ◆ **Managed but sized.** Redshift provisioned, RDS, OpenSearch, MWAA.
- ◆ **You run it.** EMR on EC2, Kafka on EC2, Airflow on EC2.
- ◆ Most services now have a serverless variant. Know which one a question means.

ON THE EXAM

Amazon EMR appears in three forms. EMR on EC2, EMR Serverless and EMR on EKS.

What each service actually charges for

SERVICE	YOU PAY FOR
Amazon S3	Data stored, requests, and retrieval from cold tiers
Amazon Athena	Bytes scanned per query. Nothing when idle
AWS Glue ETL	DPU-hours while the job runs, billed per second
AWS Lambda	Requests, plus duration times memory configured
Redshift provisioned	Nodes, whether or not anyone queries
Redshift Serverless	RPU-seconds while a query runs, plus storage
Kinesis Data Streams	Shard-hours or throughput, continuously

ON THE EXAM

Billed continuously means expensive when idle. Billed per use means expensive when constant.

Batch or streaming?

BATCH

Nobody is waiting

- Runs on a schedule or a trigger
- Reprocessing is easy. Just run it again
- Cheaper per record at volume
- Glue, EMR, Lambda, Redshift COPY

STREAMING

Someone or something is waiting

- Runs continuously, record by record
- Reprocessing needs retention and replay
- Costs more, and never stops costing
- Kinesis, MSK, Managed Service for Apache Flink

THE TIEBREAKER

Ask who is waiting. If the answer is nobody until tomorrow morning, it is a batch question.

The latency ladder

REQUIREMENT	WHAT THAT MEANS IN PRACTICE
Sub-second	In-memory. ElastiCache, MemoryDB, DynamoDB
Seconds	Kinesis Data Streams, MSK, Flink. Firehose with zero buffering
Near real time, under 15 min	Data Firehose buffering, micro-batch Glue jobs
Minutes to hours	Scheduled Glue or EMR jobs, Redshift COPY
Next day	Nightly batch. The cheapest option that meets the need

ON THE EXAM

Firehose buffers from 0 to 900 seconds. At zero it delivers in about five seconds.

A logistics company receives GPS pings from ten thousand vehicles, arriving continuously at about two thousand records per second. The pings are currently written to Amazon S3 by a custom application running on Amazon EC2 that the team maintains.

The data is used for one thing: a report produced every morning showing yesterday's route efficiency. The operations team has asked for lower cost and less to maintain. A proposal on the table is to rebuild the ingestion using Amazon Kinesis Data Streams with a Managed Service for Apache Flink application.

What should the data engineer recommend?

- | | |
|---|---|
| A Adopt the proposal. Kinesis Data Streams with a Flink application handles the throughput | B Replace the EC2 application with Amazon Data Firehose delivering directly to Amazon S3 |
| C Keep the EC2 application but move it to a larger instance type with auto scaling | D Adopt the proposal, but use Amazon MSK instead of Kinesis Data Streams |

◆ PAUSE AND DECIDE

What should the data engineer recommend?

A Kinesis Data Streams with Flink

Works, and costs more than what they have. A Flink application is a running application to operate.

C Bigger EC2 with auto scaling

Keeps every maintenance burden they asked to remove. Solves throughput, which was never the problem.

B Data Firehose direct to S3

Fully managed, no code, no servers, scales itself. Meets the only real requirement: land the data.

D Same proposal with MSK

Worse than A. MSK adds cluster management to an architecture that was already too heavy.

THE SIGNAL

Data arrives continuously. It is consumed once a day. That is a batch problem wearing streaming clothes.

The store is chosen by **access pattern**, not by how much data there is.

Access pattern decides the store

HOW THE DATA IS READ	WHERE IT BELONGS
Single item by key, huge scale	Amazon DynamoDB
Transactional rows, joins, updates	Amazon RDS or Amazon Aurora
Aggregates over millions of rows	Amazon Redshift
Occasional SQL over files in a lake	Amazon Athena over Amazon S3
Free-text or log search	Amazon OpenSearch Service
Sub-millisecond key lookups	Amazon ElastiCache or Amazon MemoryDB
Anything durable and cheap, any format	Amazon S3

ON THE EXAM

Athena and Redshift both run SQL. Query frequency separates them, not query complexity.

Data lake or data warehouse?

DATA LAKE

Schema on read

- Files in Amazon S3, any format
- Structure applied when you query
- Cheap storage, pay per query
- Good for unknown or evolving data

DATA WAREHOUSE

Schema on write

- Tables in Amazon Redshift
- Structure enforced when you load
- Costlier storage, fast repeat queries
- Good for known, curated, queried often

THE TIEBREAKER

You are rarely asked to choose. Most correct architectures use both.

What lakehouse means on AWS now

- ◆ **One catalog, many engines.** Athena, Redshift, EMR and Glue read the same tables.
- ◆ **AWS Glue Data Catalog** holds the technical metadata. Table names, columns, partitions.
- ◆ **AWS Lake Formation** holds the permissions on those tables.
- ◆ **Apache Iceberg** adds transactions, time travel and schema evolution to files.
- ◆ **SageMaker Lakehouse** is the name for that unified access across S3 and Redshift.

ON THE EXAM

Version 1.1 of the exam guide added open table formats and business catalogs. Older banks miss both.

Why file format is a whole topic

ROW-BASED

CSV, JSON, Avro

- Reads every column to get one
- Good for writes and whole-record access
- Larger on disk, more bytes scanned
- Avro handles schema evolution well

COLUMNAR

Parquet, ORC

- Reads only the columns you asked for
- Compresses far better, similar values adjacent
- Good for analytics, poor for row writes
- The default answer for query cost questions

THE TIEBREAKER

Athena and Spectrum charge by bytes scanned. Columnar plus partitioning attacks the bill directly.

The optimisation stack, in order of impact

- ◆ **Partition** on what you filter by. Usually date. Skips whole prefixes.
- ◆ **Convert to columnar**. Parquet by default, ORC if the question says so.
- ◆ **Compress**. Snappy for speed, gzip for size. Splittable matters for Spark.
- ◆ **Compact small files**. Thousands of tiny objects cripple every engine.
- ◆ Over-partitioning is a real failure. Too many tiny partitions is as bad as none.

ON THE EXAM

Small files are the most common real cause of a slow lake, and a frequent exam answer.

Vocabulary for reliability questions

- ◆ **Idempotent.** Running it twice produces the same result as once.
- ◆ **Replayability.** Can you reprocess from the source after a failure?
- ◆ **At-least-once.** Nothing is lost. Duplicates are possible.
- ◆ **Exactly-once.** No loss, no duplicates. Costs more, needs state.
- ◆ **Stateless.** Each record processed alone. **Stateful.** Needs what came before.

ON THE EXAM

Firehose is at-least-once, except to S3, Iceberg Tables and Snowflake.

A media company stores viewing events as compressed JSON files in Amazon S3, partitioned by ingestion date. Analysts query them with Amazon Athena. A new customer-facing feature needs to show a single viewer their own watch history when they open the app.

The feature must respond in under one hundred milliseconds, and will be called several thousand times per second at peak. The team's first attempt used Athena and the page timed out.

Why did the first attempt fail, and what should replace it?

- | | |
|--|---|
| <p>A Athena was not partitioned correctly.
Repartition by viewer ID and keep using Athena</p> | <p>B Convert the JSON files to Apache Parquet, which will bring the query under the latency target</p> |
| <p>C Serve the feature from Amazon DynamoDB, keyed on viewer ID, populated from the same events</p> | <p>D Load the events into Amazon Redshift and query it from the application</p> |

◆ PAUSE AND DECIDE

Why did the first attempt fail, and what should replace it?

- A

 Repartition by viewer ID
Would create millions of tiny partitions, and Athena still starts queries in seconds, not milliseconds.
- B

 Convert to Parquet
The right optimisation for the wrong problem. It reduces cost and scan time, not per-query startup latency.
- C

 DynamoDB keyed on viewer ID
Single-item lookup by key, single-digit millisecond, scales to thousands per second. Matches the access pattern.
- D

 Load into Redshift
An analytical warehouse serving per-user application reads. Faster than Athena, still the wrong shape.

THE SIGNAL

Compare with scenario 1. There, streaming was overkill. Here, an analytical store is underpowered. Same question, opposite direction.

How work gets started

SCHEDULED

Time-based

- EventBridge Scheduler, cron expressions
- Glue triggers, MWAA schedules
- Predictable, easy to reason about
- Runs whether data arrived or not

EVENT-DRIVEN

Something happened

- S3 Event Notifications, EventBridge rules
- Fires when the file actually lands
- Lower latency, no wasted empty runs
- Harder to reason about at scale

THE TIEBREAKER

"As soon as the file arrives" is an event trigger. "Every night" is a schedule.

Four questions to ask of every exam question

In this order

- ◆ **Who is waiting?** Nobody until morning means batch.
- ◆ **How is it read?** By key, aggregated, or searched. That picks the store.
- ◆ **What is the qualifier?** Overhead, cost, latency, or effort.
- ◆ **What did they rule out?** A stated constraint eliminates before you compare.
- ◆ Two of the four options usually die on question four alone.

ON THE EXAM

Next: Domain 1 begins with batch ingestion and the AWS Glue ETL engine.